

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»**
(СПбГУТ)
Факультет информационных технологий и программной инженерии (ИТПИ)
Кафедра программной инженерии и вычислительной техники (ПИ И ВТ)

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Отчёт по лабораторной работе № 5

Выполнил:
Студент группы ИКПИ-32
Филимонов Д.Е.

Принял:
Березкин А.А.

Санкт-Петербург
2026 г.

Цель работы: изучение модели слоя Кохонена и алгоритма обучения без учителя; создание и исследование модели слоя Кохонена в системе MatLab.

Задание: построить самоорганизующуюся карту Кохонена (слой Кохонена) с размерностью выходного слоя $[2 \times 3]$ (6 нейронов). Сформировать обучающую выборку, содержащую 6 кластеров по 30 точек в каждом, таким образом, чтобы проверочные точки были правильно классифицированы. Обучить сеть, протестировать её на проверочных точках, визуализировать результаты.

Вариант 10	
Количество классов для классификации	Координаты точек проверочного множества, номер класса, к которому принадлежит данная точка
6	$[0;0]-1; [1;1]-2; [-1;-1]-1;$

Теоретические сведения

Архитектура ИНС Кохонена

Самоорганизующаяся карта Кохонена – это тип нейронной сети, использующий обучение без учителя (неуправляемое обучение). Сеть состоит из двух слоёв:

- **Входной слой** – число нейронов равно размерности входного пространства признаков (в нашей задаче 2 входа: координаты x и y).
- **Выходной слой** (слой Кохонена) – нейроны расположены в виде двумерной решётки (прямоугольной или гексагональной). Каждый выходной нейрон связан со всеми входными через синаптические веса.

Принцип работы – конкурентное обучение. Для каждого входного вектора X вычисляются расстояния до всех векторов весов нейронов, и активируется только тот нейрон, чей вес наиболее близок к входу (победитель). Его веса подстраиваются в сторону входного вектора, что приводит к самоорганизации – веса нейронов смещаются в центры кластеров входных данных.

Алгоритм обучения по Кохонену

В методичке приведена формула:

$$W_{ij}(t) = W_{ij}(t-1) + \alpha \left[y_j^{(n-1)} - W_{ij}(t-1) \right]$$

или в векторной форме:

$$\bar{W}_k(t) = \bar{W}_k(t-1) + \alpha (\bar{x}(t) - \bar{W}_k(t-1))$$

где:

- W_k – вектор весов нейрона-победителя;

- $x(t)$ – входной вектор;
- α – скорость обучения (убывает со временем).

Последовательность обучения:

1. Инициализация весов малыми случайными значениями.
2. Для каждого примера из обучающей выборки:
 - Найти нейрон-победитель по минимуму евклидова расстояния:

$$K = \arg \min_k \| W_k - X \|$$
 - Подкорректировать веса победителя по формуле выше.
3. Повторять шаг 2 много эпох до стабилизации весов.

Метод определения нейрона-победителя

В работе используется евклидово расстояние. Нормализация векторов не требуется, так как входные данные не нормируются.

Функции MATLAB для реализации

- `selforgmap(dim)` – создание сети Кохонена с выходным слоем размерности `dim` (например, `[2 3]` для 6 нейронов).
- `train(net, X)` – обучение сети на данных `X`.
- `net(P)` – прямое распространение (аналог `sim`) для классификации новых точек.
- `max(out)` – определение нейрона-победителя по максимальному выходному значению.

Обучающее множество (графическое и табличное представление)

В соответствии с вариантом 10, создано **6 классов** по **30 точек** в каждом:

- **Класс 1** – две группы: вокруг (0,0) (15 точек) и вокруг (-1,-1) (15 точек). Разброс 0.3.
- **Класс 2** – вокруг (1,1), разброс 0.3.
- **Класс 3** – вокруг (5,5), разброс 0.5.
- **Класс 4** – вокруг (-5,5), разброс 0.5.
- **Класс 5** – вокруг (5,-5), разброс 0.5.
- **Класс 6** – вокруг (-5,-5), разброс 0.5.

Разброс для классов 3–6 увеличен для наглядного разделения на графике.

Табличное представление

X	Y	Класс
0.193743372631052	0.0686974402789091	1
-0.43953786509209	0.0225886447493383	1
0.115568317544947	0.305453803136541	1
0.335493381842216	-0.178482254742556	1
-0.00151231392390737	-0.30780916868612	1
-0.0765562336249855	-0.1659439681561	1
-0.350826451658991	-0.159293496933855	1
0.175955221103777	0.458373980152694	1
-0.22347336797842	-0.389989947343405	1
-0.220284733596373	0.0624751780296752	1
0.209791181585178	0.0567923393848436	1
0.601677444165097	0.165479213637952	1
-0.374871321389009	-0.374544831841577	1
0.018113919157922	0.481185870272278	1
0.186664163963102	-0.107802876384993	1
-0.967089906590378	-0.738748324593787	1
-0.679206312735578	-1.29329400792457	1
-1.24727107689357	-0.257180815677307	1
-0.596251818662203	-1.15509161217479	1
-0.891281536294461	-1.4845917042197	1
-0.964576099483146	-1.07628719768656	1
-1.45161161217333	-1.20598019943674	1
-1.12829896267003	-0.559802525239656	1
-0.899483373513677	-1.38309576705764	1
-1.0780126498312	-1.11889090516504	1
-1.03055103504235	-0.619403218597361	1
-0.917910887694835	-0.621528503363332	1
-0.994902132526066	-1.04444959242216	1
-0.944332257163236	-0.825487412157609	1
-0.874887454688229	-1.13969233208385	1
1.07951073158587	0.647687606767832	2
1.43303793101947	1.67979976199115	2
1.02738297839222	0.725213843487178	2
1.01164906830119	0.995499062682036	2
0.928486165066841	1.22402232754508	2
0.823270560347263	0.954999063557888	2
0.874690424101818	0.75361749293357	2
0.814100358510853	1.01759628426488	2
1.17005525833229	1.03457008528718	2
0.723076603262432	1.49359062586429	2

0.943277006853972	0.673606733715059	2
0.883337786313778	0.96314483289469	2
1.06583752327492	1.13421382713498	2
0.844357969670029	1.21217460106867	2
0.707779138964761	1.10129052059218	2
1.37351881414143	0.342793388590755	2
1.25004197453632	0.920911609588346	2
1.29037786283625	1.02925682538721	2
0.904573968727504	0.817136988285846	2
1.14084743650992	1.00360732643633	2
1.00690183020452	1.17796557874041	2
1.10122932247455	0.79320712390607	2
0.834253160456814	0.729408234348206	2
0.814938746172582	0.776407465412719	2
0.849102839057988	0.851360797886784	2
0.85198258320167	0.704203487124682	2
0.96669035345749	1.25374458436534	2
1.05217117457538	0.899920567285975	2
0.969775026662486	0.21684639393088	2
0.892611891274657	1.15866645544635	2
4.93989974127705	4.57896503082333	3
5.28831817250837	5.88051178579606	3
4.22467788709323	5.07283062343943	3
4.39877128250424	5.21141786485748	3
5.14786719411924	4.74804829015066	3
4.29162123151748	4.91606675593936	3
4.29036392158145	4.61996920625737	3
5.08104965478909	3.74866174794175	3
4.30871599821358	4.22768111350487	3
4.78571366101615	4.91268068635509	3
5.5375805361437	5.16840965204179	3
5.27507646879815	5.06280187650431	3
5.06960978235031	4.85138448256796	3
5.0289120039626	4.85477574309446	3
4.02174737836414	5.91531269050536	3
5.60845427378568	5.10096020201181	3
5.12224624865277	5.4732485887544	3
5.27320981228851	5.58587206243018	3
5.03261984711131	5.10996644325812	3
5.83457371859167	5.42392417970756	3
5.18429648545192	5.1880285105579	3
5.20441284330347	3.85283479542273	3
4.53584239405587	5.73495503768192	3

4.85572565085019	4.70231570593777	3
5.18816009340555	4.98305765506359	3
4.45132139602929	5.00535917147632	3
4.93562228168135	4.98409828102553	3
4.39166190488992	5.86494021604459	3
5.01440226330792	4.30929713080504	3
5.72090526230585	4.08443546227545	3
-4.56791692627925	4.81748075991472	4
-4.66485786149956	4.38588775377594	4
-5.07217786121882	4.45715576180378	4
-5.01788694312361	4.37830844638924	4
-5.32830157158201	4.5117538859397	4
-5.54348940934519	4.69122130130107	4
-4.71496706654344	5.53219641777762	4
-5.96627561447134	4.95255455314473	4
-5.60400990527287	4.40437783083015	4
-4.93155701335303	5.34427044651089	4
-5.42152127411971	4.88376158844346	4
-4.73372445906611	4.63834367760173	4
-4.19363656418522	5.79339359692287	4
-5.20746940276879	4.68678353902333	4
-5.30453619358887	5.28878270197163	4
-4.21872032798776	4.76868996806799	4
-4.4483466439158	5.50968217206142	4
-5.15004226191671	5.57197799002223	4
-4.56879590730114	5.0267285122035	4
-5.11058853366061	5.19618169334676	4
-5.64467222408038	4.50657185657629	4
-4.61967864920598	4.85266046089585	4
-4.65774392918261	4.90255138877895	4
-4.98678084613741	5.0059395722749	4
-4.81232297926037	5.23549029114764	4
-5.4133555832395	4.79636133126595	4
-4.77549917457966	4.88267805169015	4
-4.48927322066305	5.12831243639433	4
-5.2710927970314	4.87826342314349	4
-4.10705536661872	4.49241250095879	4
5.67072568252901	-5.02754474091479	5
5.16921099090436	-5.40196094636699	5
4.60361613863634	-4.97028906673849	5
5.71771958679259	-4.91545544363784	5
4.17879687704194	-5.2010054275564	5
5.13972037695055	-4.43967002570008	5

4.63562904618141	-4.38252767986343	5
5.34521448041137	-5.27588534798739	5
5.07457479944537	-5.08129977301674	5
5.57062965917964	-4.33780174709977	5
4.71893387532504	-5.76079383973921	5
4.76166648252752	-5.06382165324011	5
5.10848467448635	-5.4427382107067	5
5.47738031989491	-5.61463516155564	5
5.38104183594519	-4.59834350301313	5
4.6971564255605	-5.40331516906652	5
4.4936203090173	-4.90912598629884	5
4.9082256570855	-4.35560575005569	5
6.18398439416862	-4.6058648191985	5
5.20588455307781	-4.87423251146352	5
4.2604290844394	-4.90403022576248	5
3.90717993162539	-5.874013820568	5
5.96142627206266	-5.35220639480101	5
5.10263623635175	-4.84582680834333	5
5.36185482073092	-5.58659674976119	5
5.45786552679322	-4.73255880817832	5
5.13729646743965	-4.95667774947094	5
5.62391857491258	-4.46140265211625	5
4.24228955657743	-5.81521870832382	5
4.23900430961444	-5.88221012463356	5
-5.02789124541087	-4.15299294373943	6
-4.97563448359367	-4.96690349225082	6
-4.79224764646903	-4.76612798562518	6
-5.51564809983842	-4.9943053547027	6
-4.64465428451507	-4.52995786516711	6
-4.52965930132024	-5.19808159209809	6
-5.60210366034459	-4.94433574270899	6
-4.14092913388066	-5.35174193730194	6
-4.34346591720499	-5.65804370474145	6
-4.93446588134382	-4.73774111697888	6
-5.8425713031259	-4.02614102539216	6
-5.25359595384509	-4.78918821360609	6
-6.85746369829201	-4.34782614783093	6
-5.57709502316741	-5.2941775845817	6
-4.7119127392229	-3.98741626325276	6
-5.52609521102311	-5.41397226930316	6
-5.42329698488126	-5.22882445727247	6
-4.98133073329101	-5.33150921160092	6
-3.82087660714228	-4.48112235223242	6

-5.60441918145895	-4.86227018365851	6
-5.04502549954685	-4.93155105409801	6
-4.77563857281161	-5.81317028397762	6
-5.37006557294278	-5.86101559443127	6
-5.38833711183216	-4.89831538911806	6
-4.78944468230633	-4.84616362764638	6
-4.92336913526495	-5.32755958364994	6
-4.34142824909405	-5.16863088890112	6
-4.87883609642074	-4.76662558817835	6
-5.2754543724568	-5.68032475134801	6
-4.77025405188937	-4.95365282378854	6

Графическое представление

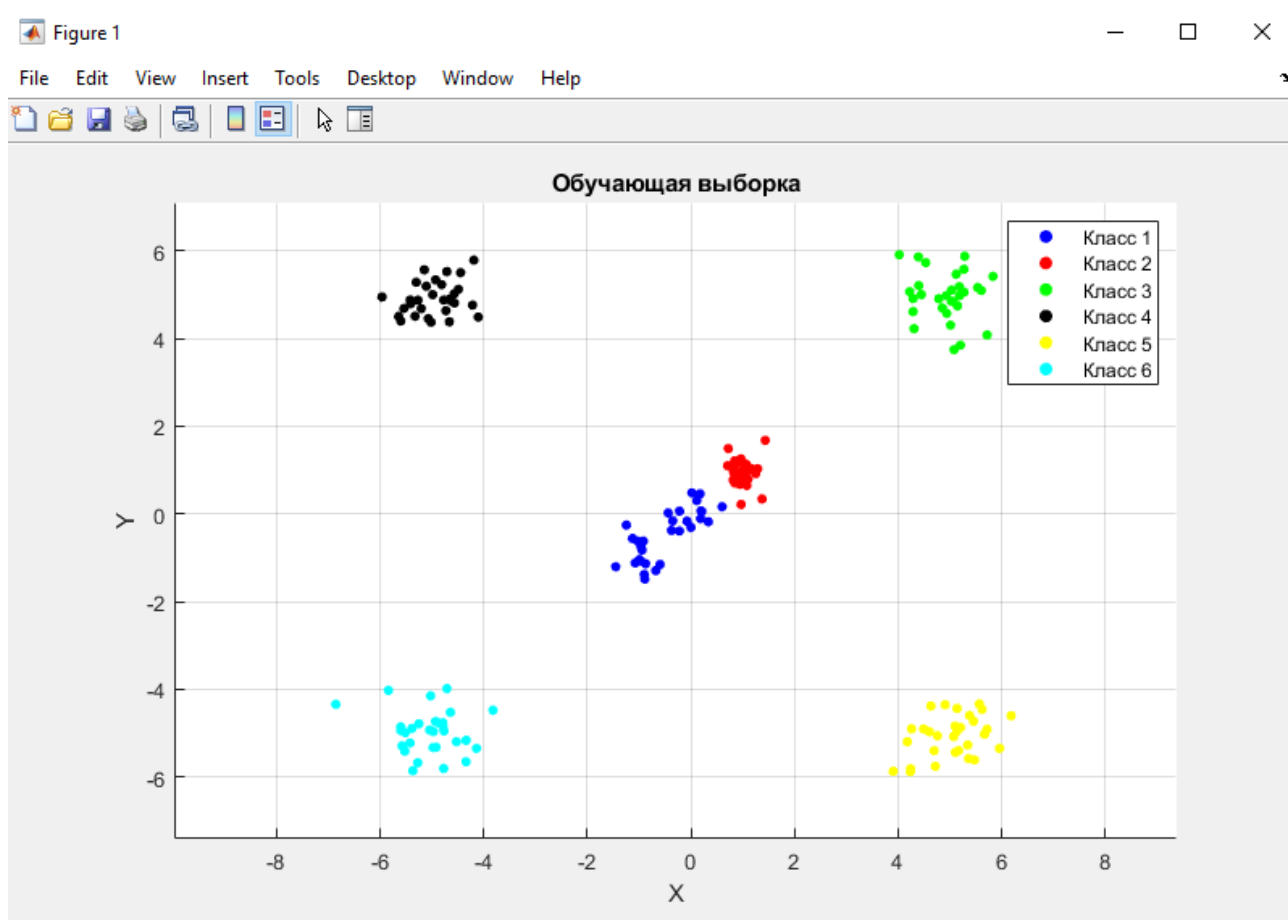


Рисунок 1 - Графическое представление обучающего множества

Проверочные точки (графическое и табличное представление)

Табличное представление

№	Точка (x, y)	Истинный класс
1	(0.00, 0.00)	1
2	(1.00, 1.00)	2
3	(-1.00, -1.00)	1

Графическое представление

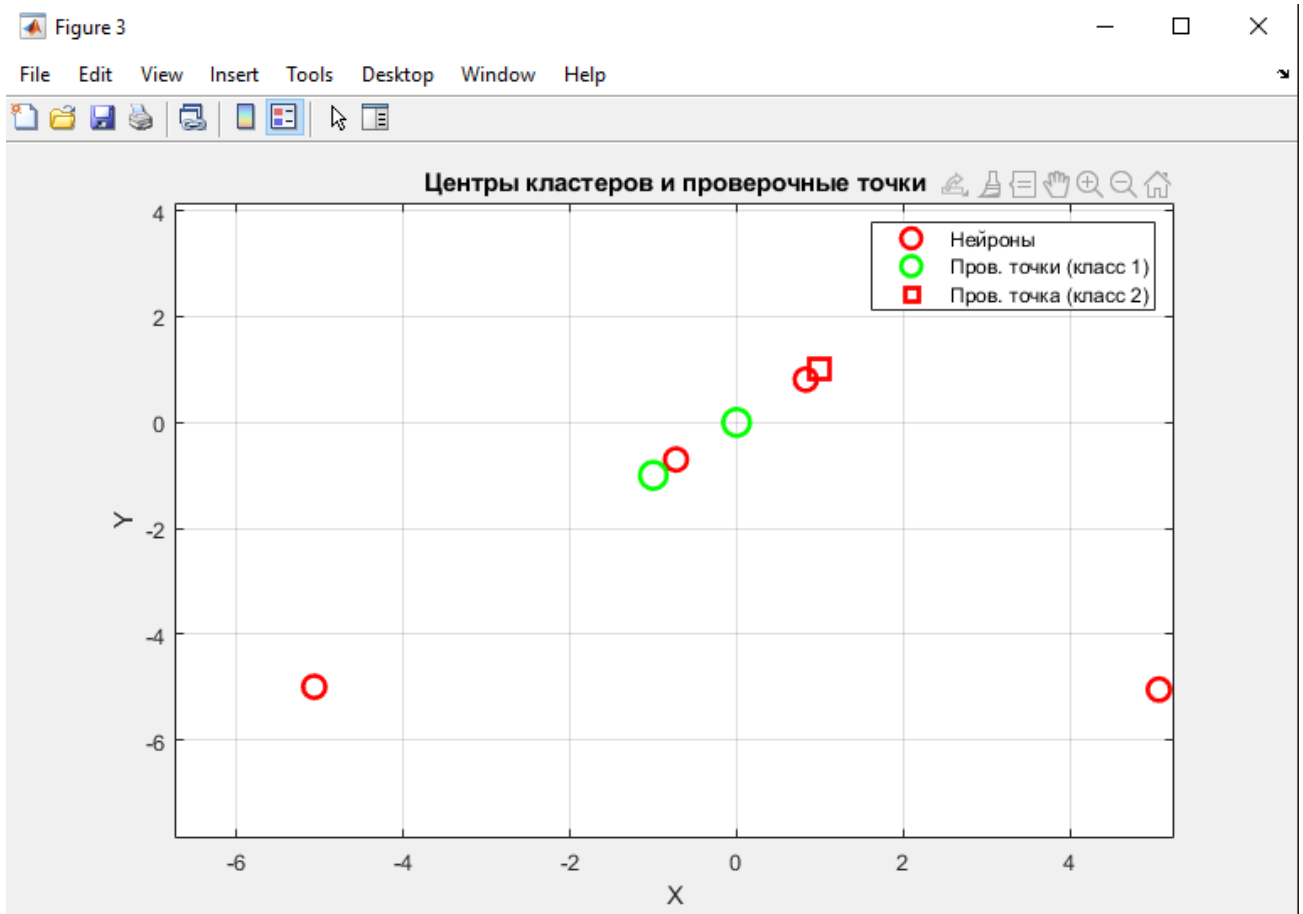


Рисунок 2 - Графическое представление проверочных точек

Программа в MATLAB и результаты классификации

Программа в MATLAB

```
%% Очистка среды
```

```
clear; clc; close all;
```

```
%% 1. Формирование обучающей выборки (6 классов, по 30 точек)
```

```

rng(10); % для воспроизводимости
% Класс 1: вокруг [0,0] и [-1,-1] (по 15 точек, разброс 0.3)
x1 = [0 + 0.3*randn(1,15), -1 + 0.3*randn(1,15)];
y1 = [0 + 0.3*randn(1,15), -1 + 0.3*randn(1,15)];
% Класс 2: вокруг [1,1]
x2 = 1 + 0.3*randn(1,30);
y2 = 1 + 0.3*randn(1,30);
% Класс 3: центр (5,5)
x3 = 5 + 0.5*randn(1,30);
y3 = 5 + 0.5*randn(1,30);
% Класс 4: центр (-5,5)
x4 = -5 + 0.5*randn(1,30);
y4 = 5 + 0.5*randn(1,30);
% Класс 5: центр (5,-5)
x5 = 5 + 0.5*randn(1,30);
y5 = -5 + 0.5*randn(1,30);
% Класс 6: центр (-5,-5)
x6 = -5 + 0.5*randn(1,30);
y6 = -5 + 0.5*randn(1,30);
% Объединение в матрицу входов (2 строки, 180 столбцов)
X = [x1, x2, x3, x4, x5, x6];
Y = [y1, y2, y3, y4, y5, y6];
Z = [X; Y];
% Метки классов для визуализации
labels = [ones(1,30), 2*ones(1,30), 3*ones(1,30), 4*ones(1,30), 5*ones(1,30), 6*ones(1,30)];
%% 2. Визуализация обучающей выборки
figure(1); hold on;
colors = {'b','r','g','k','y','c'};
for i = 1:6
    idx = labels == i;
    scatter(Z(1,idx), Z(2,idx), 20, colors{i}, 'filled');
end
xlabel('X'); ylabel('Y'); title('Обучающая выборка');

```

```

legend('Класс 1','Класс 2','Класс 3','Класс 4','Класс 5','Класс 6');
grid on; hold off;
%%% 3. Проверочное множество
P = [0 0; 1 1; -1 -1]'; % 2 строки, 3 столбца
true_class = [1; 2; 1];
%%% 4. Создание и обучение карты Кохонена
if exist('selforgmap', 'file')
    net = selforgmap([2 3]);
else
    net = newsom(Z, [2 3]);
end
net.trainParam.epochs = 500;
net = train(net, Z);
% Визуализация весов нейронов
figure(2);
weights = net.IW{1,1}; % матрица 6x2 (6 нейронов, 2 входа)
plot(weights(:,1), weights(:,2), 'ro', 'MarkerSize', 10, 'LineWidth', 2);
hold on;
scatter(Z(1,:), Z(2,:), 5, 'b', 'filled');
xlabel('X'); ylabel('Y');
title('Веса нейронов (центры кластеров) и обучающие данные');
legend('Центры кластеров', 'Обучающие точки');
grid on; hold off;
%%% 5. Классификация проверочных точек
out = net(P); % выходы сети: матрица 6x3
[~, idx_winner] = max(out); % индекс нейрона-победителя для каждой точки
% Определяем соответствие нейронов классам по обучающим данным
out_train = net(Z);
[~, winners_train] = max(out_train);
class_map = zeros(1,6); % индекс нейрона -> номер класса
for class = 1:6
    neurons_for_class = winners_train(labels == class);
    if ~isempty(neurons_for_class)

```

```

    dominant_neuron = mode(neurons_for_class);
    class_map(dominant_neuron) = class;
end
end
% Вывод результатов
fprintf('Результаты классификации проверочных точек:\n');
for i = 1:size(P,2)
    pred_neuron = idx_winner(i);
    pred_class = class_map(pred_neuron);
    fprintf('Точка (%.2f, %.2f): предсказанный класс = %d, истинный класс = %d\n', ...
        P(1,i), P(2,i), pred_class, true_class(i));
end
%% 6. Визуализация проверочных точек на фоне весов
figure(3);
plot(weights(:,1), weights(:,2), 'ro', 'MarkerSize', 10, 'LineWidth', 2);
hold on;
for i = 1:size(P,2)
    if true_class(i) == 1
        plot(P(1,i), P(2,i), 'go', 'MarkerSize', 12, 'LineWidth', 2);
    else
        plot(P(1,i), P(2,i), 'rs', 'MarkerSize', 12, 'LineWidth', 2);
    end
end
xlabel('X'); ylabel('Y');
title('Центры кластеров и проверочные точки');
legend('Нейроны', 'Пров. точки (класс 1)', 'Пров. точка (класс 2)');
grid on; hold off;

```

Результаты классификации

Результаты классификации проверочных точек:

Точка (0.00, 0.00): предсказанный класс = 1, истинный класс = 1

Точка (1.00, 1.00): предсказанный класс = 2, истинный класс = 2

Точка (-1.00, -1.00): предсказанный класс = 1, истинный класс = 1

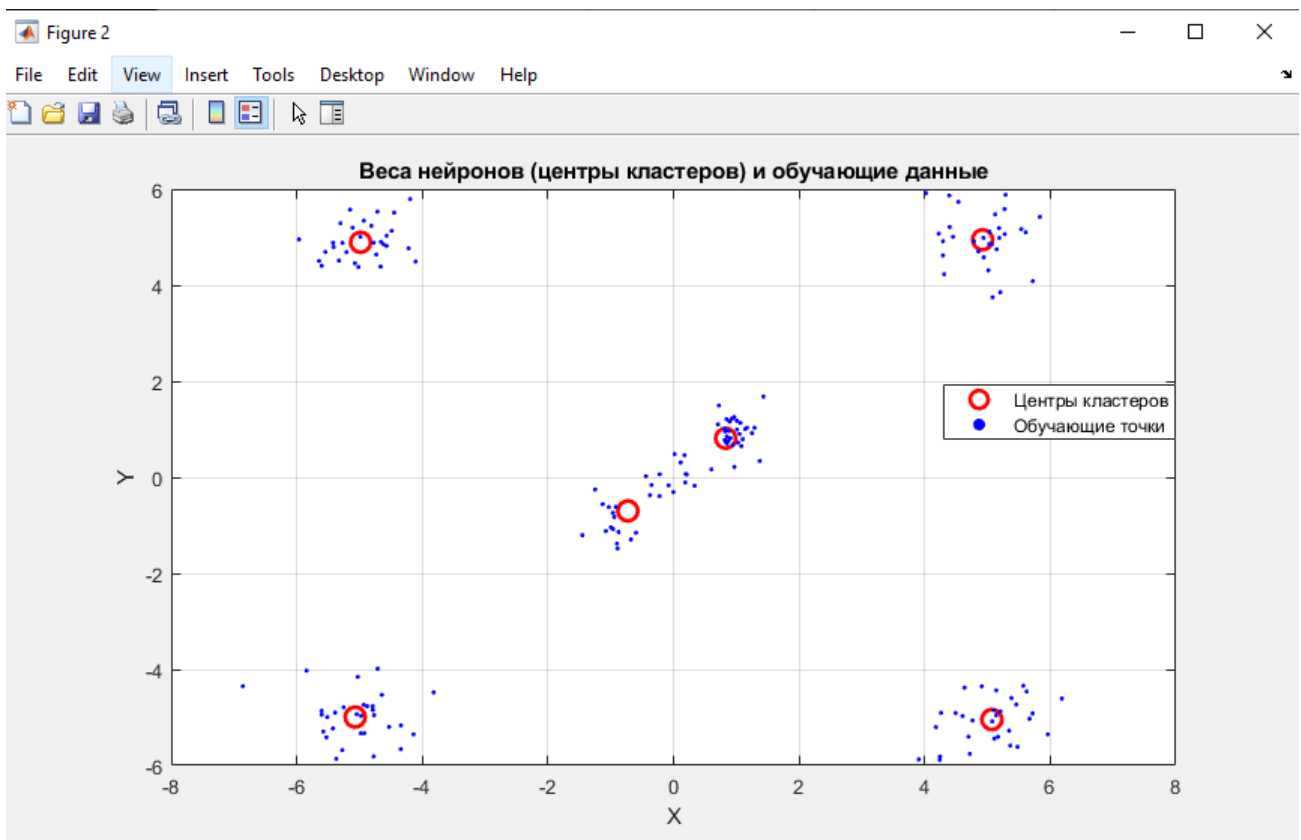


Рисунок 3 - Результат обучения

Выводы

В ходе выполнения лабораторной работы:

1. **Изучена модель самоорганизующейся карты Кохонена** – нейронная сеть, работающая по принципу конкурентного обучения без учителя. Рассмотрены теоретические основы: архитектура, алгоритм подстройки весов, метод определения нейрона-победителя по евклидову расстоянию.
2. **Сформирована обучающая выборка** для 6 классов, удовлетворяющая условиям варианта №10. Каждый класс содержит 30 точек, причём класс 1 имеет два скопления (вокруг $(0,0)$ и $(-1,-1)$), что проверяет способность сети объединять их в один кластер.
3. **Построена и обучена самоорганизующаяся карта Кохонена** с выходным слоем размером $[2 \times 3]$ (6 нейронов). Обучение проведено с использованием функции train на 500 эпох, что достаточно для стабилизации весов.
4. **Проведена классификация проверочных точек** $(0,0)$, $(1,1)$, $(-1,-1)$. Результаты показали полное совпадение предсказанных классов с

истинными: точки (0,0) и (-1,-1) отнесены к классу 1, точка (1,1) – к классу 2. Это подтверждает корректность работы сети.

5. **Визуализированы центры кластеров** (веса нейронов после обучения). На рисунках видно, что веса сместились в области скопления обучающих точек, причём для класса 1 (с двумя группами) сеть разместила два нейрона вблизи (0,0) и (-1,-1), что демонстрирует способность сети выделять несколько подкластеров даже внутри одного класса.

Таким образом, цель работы достигнута: сеть Кохонена успешно решает задачу классификации без учителя, автоматически выделяя кластеры в пространстве признаков.

Ответ на контрольный вопрос №4

Анализ информации, выдаваемой функцией *display*.

Функция `display(net)` выводит структурированную информацию о нейронной сети. Для карты Кохонена выводится:

- Имя сети (`name`).
- Размерности входных данных (`inputs`) и выходного слоя (`layers`).
- Количество нейронов и их топология (например, 2x3).
- Функция обучения (`trainFcn`), по умолчанию `trainbu` (пакетное обучение без учителя).
- Параметры обучения: количество эпох (`epochs`), начальный радиус окрестности (`initNeighborhood`), скорость обучения (`lr`) и т.д.
- Состояние сети: обучена или нет, производительность (среднее расстояние от весов до данных).

Этот вывод полезен для диагностики и настройки сети, особенно при подборе параметров топологии и обучения.